Clase 13 (opcional/bonus) — Frontend mínimo para mostrar TaskFlow

Duración estimada: 2-3 horas | No forma parte de las 12 clases obligatorias

Cómo leer esta guía

Combina **conceptos** (qué es y por qué) con **práctica** (cómo), y cierra con preguntas de repaso.

Parte 0 — Por qué esta clase existe

Las 12 clases construyen exclusivamente el **backend** (Clase 1, Parte 1.4: la parte que corre en el servidor). Nunca vimos una pantalla "linda" porque no era el objetivo de la materia. Esta clase opcional agrega una página web mínima, en HTML/JavaScript puro (sin frameworks de frontend, sin instalar nada nuevo), que consume la API de TaskFlow. Es útil para mostrar el trabajo final de forma visual, en una demo o presentación.

> 💡 **Nota del docente**
> Herramientas usadas: solo HTML, CSS y JavaScript "vanilla" (sin librerías), que corren nativamente en cualquier navegador: 100% gratuito, sin instalación adicional. No se agrega ningún paquete nuevo a `requirements.txt`.

Concepto: frontend y backend separados. Durante la materia construimos una **API**: el backend no genera pantallas, solo recibe y devuelve datos en JSON. Quien dibuja la pantalla es otro programa, el **frontend**, que corre en el navegador del usuario. Esta separación permite que la misma API sirva a una página web, a una app móvil, a otros servicios (la comisión de los jueves) y a una IA.

Hay dos grandes formas de armar una aplicación web:

Enfoque	Quién arma el HTML	Ejemplos	Ventajas	Desventajas
Renderizado en el servidor	El backend, en cada petición	Templates de Django, el `/admin/`	Simple; buena indexación en buscadores	Cada acción recarga la página
Aplicación de una sola página (SPA)	El JavaScript del navegador, con datos de una API	Esta clase; React, Vue, Angular	Interfaz fluida, sin recargas; el backend es solo una API	Más código en el cliente; hay que resolver autenticación y CORS

Hoy hacemos una SPA mínima, sin frameworks, para ver el mecanismo sin intermediarios. Los frameworks de frontend hacen lo mismo que vamos a hacer a mano, a mayor escala.

Parte 1 — Preparar el backend para recibir al frontend (20 min)

1.1 Confirmar CORS

Repasen la Clase 8, Parte 3.1: el navegador solo deja que la página lea las respuestas de la API si su **origen** figura en `CORS_ALLOWED_ORIGINS`. Vamos a servir la página en `http://127.0.0.1:5500` (Parte 3), así que agréguenlo en `taskflow/config/settings.py`:

```
CORS_ALLOWED_ORIGINS = [
    "http://localhost:3000",
    "http://127.0.0.1:5500",
    "https://taskflow-app.com",
]
```

Ojo: para el navegador, `http://127.0.0.1:5500` y `http://localhost:5500` son **orígenes distintos**. Abran la página exactamente con la dirección que pusieron en la lista.

1.2 Levantar el backend

Parados en `taskflow/`, entorno virtual activo:

```
python manage.py runserver
```

Déjenlo corriendo en una terminal aparte durante toda esta clase. Necesitan también un usuario que sea **admin** de al menos un proyecto (Clase 7) para poder crear tareas desde la página.

Parte 2 — La página (45 min)

2.1 Crear la carpeta del frontend

Fuera de `taskflow/` (al lado, no adentro, para no mezclar backend y frontend), parados en `programacion4/` (la carpeta de la Clase 1, Parte 2.3):

```
mkdir taskflow-frontend
cd taskflow-frontend
```

A partir de acá, todos los archivos de esta clase van dentro de `taskflow-frontend/`.

2.2 El archivo `index.html`

Creen `taskflow-frontend/index.html`:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>TaskFlow</title>
  <style>
    body { font-family: sans-serif; max-width: 700px; margin: 40px auto; padding: 0 16px; }
    input, select, button { padding: 8px; margin: 4px 0; }
    #tasks li { margin: 6px 0; }
    .error { color: red; }
  </style>
</head>
<body>
  <h1>TaskFlow</h1>

  <section id="login-box">
    <h2>Ingresar</h2>
    <input id="username" placeholder="usuario">
    <input id="password" type="password" placeholder="contraseña">
    <button onclick="login()">Ingresar</button>
    <p id="login-error" class="error"></p>
  </section>

  <section id="app-box" style="display:none;">
    <h2>Mis tareas</h2>
    <ul id="tasks"></ul>
    <select id="project"></select>
    <input id="new-title" placeholder="Nueva tarea">
    <button onclick="createTask()">Agregar</button>
    <p id="app-error" class="error"></p>
  </section>

  <script src="app.js"></script>
</body>
</html>
```

La estructura tiene dos secciones: `login-box` (visible al principio) y `app-box` (oculta con `display:none` hasta que el usuario se loguee). El `<select id="project">` es un desplegable que vamos a llenar con los proyectos del usuario, para no tener que escribir el `id` a mano. `<script src="app.js">` va al final del `<body>` para que, cuando se ejecute, los elementos de la página ya existan.

2.3 El archivo `app.js`

Concepto: JavaScript asincrónico. En el navegador, igual que en el servidor ASGI de la Clase 9, hay **un solo hilo** que atiende todo: si esperara bloqueado la respuesta de la API, la página se congelaría. Por eso las operaciones de red son **asincrónicas**:

- `fetch(url, opciones)` inicia una petición HTTP y devuelve enseguida una **promesa** (*Promise*): un objeto que representa un resultado que todavía no llegó.
- `await` (dentro de una función `async`) espera esa promesa **sin congelar la página**: mientras tanto, el navegador sigue respondiendo a clics y dibujando.
- Es exactamente el mismo modelo `async`/`await` de Python que usamos en los consumers de Channels (Clase 9): mismo concepto, otro lenguaje.

Dos detalles de `fetch` que suelen confundir:

- `fetch` **no** falla ante un `401`, `403` o `500`: la promesa se resuelve igual, con esa respuesta. Solo falla si no hubo respuesta (sin red, servidor caído, o bloqueo de CORS). Por eso el código revisa `response.ok` (que vale `true` solo para los códigos 2xx).
- El cuerpo se lee aparte, y también es asincrónico: `await response.json()`.

Concepto: el DOM. El **DOM** (*Document Object Model*) es la representación del HTML como objetos que JavaScript puede leer y modificar: `document.getElementById("tasks")` obtiene la lista, `createElement` crea un elemento nuevo y `appendChild` lo agrega. Cambiar el DOM es lo que hace que la pantalla se actualice sin recargar.

Práctica.

Creen `taskflow-frontend/app.js`:

```
const API_URL = "http://127.0.0.1:8000/api";
let accessToken = null;

function authHeaders() {
  return { Authorization: `Bearer ${accessToken}` };
}

async function login() {
  const username = document.getElementById("username").value;
  const password = document.getElementById("password").value;

  const response = await fetch(`${API_URL}/token/`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username, password }),
  });

  if (!response.ok) {
    document.getElementById("login-error").textContent =
      response.status === 429
        ? "Demasiados intentos. Esperen un minuto."
        : "Usuario o contraseña incorrectos.";
    return;
  }

  const data = await response.json();
  accessToken = data.access;

  document.getElementById("login-box").style.display = "none";
  document.getElementById("app-box").style.display = "block";
  loadProjects();
  loadTasks();
}

async function loadProjects() {
  const response = await fetch(`${API_URL}/projects/`, { headers: authHeaders() });
  const data = await response.json();

  const select = document.getElementById("project");
```

```
  select.innerHTML = "";
  data.results.forEach((project) => {
    const option = document.createElement("option");
    option.value = project.id;
    option.textContent = project.name;
    select.appendChild(option);
  });
}

async function loadTasks() {
  const response = await fetch(`${API_URL}/tasks/`, { headers: authHeaders() });
  const data = await response.json();

  const list = document.getElementById("tasks");
  list.innerHTML = "";
  data.results.forEach((task) => {
    const li = document.createElement("li");
    li.textContent = `${task.title} — ${task.status}`;
    list.appendChild(li);
  });
}

async function createTask() {
  const title = document.getElementById("new-title").value;
  const project = document.getElementById("project").value;
  const errorBox = document.getElementById("app-error");
  errorBox.textContent = "";
  if (!title || !project) return;

  const response = await fetch(`${API_URL}/tasks/`, {
    method: "POST",
    headers: { "Content-Type": "application/json", ...authHeaders() },
    body: JSON.stringify({ title, project: Number(project) }),
  });

  if (!response.ok) {
    const data = await response.json();
    errorBox.textContent = JSON.stringify(data.detail);
    return;
  }

  document.getElementById("new-title").value = "";
  loadTasks();
}
```

Repaso de lo que hace cada función:

- `authHeaders()` arma el header `Authorization: Bearer ...` (Clase 6). Lo separamos en una función porque lo usan tres peticiones distintas.
- `login()` reproduce, desde JavaScript, la misma petición `curl` de la Clase 6 contra `/api/token/`, y guarda el `access` token en la variable `accessToken`. Queda en la memoria de la página: se pierde al recargar (mejora posible, ver la tarea). Si la respuesta es `429` (el límite de 5 intentos por minuto de la Clase 8), muestra un mensaje específico.
- `loadProjects()` pide `/api/projects/` y llena el desplegable. Gracias a la Clase 7, la API solo devuelve los proyectos donde el usuario es miembro.
- `loadTasks()` pide el listado de tareas y lo muestra. La respuesta viene **paginada** (Clase 5), por eso recorremos `data.results`: se ven las 5 primeras tareas (el `PAGE_SIZE`).
- `createTask()` hace el mismo `POST /api/tasks/` de la Clase 4, con el proyecto elegido en el desplegable. `Number(project)` convierte el valor del `<select>` (que siempre es texto) en número. Si la API responde con error, por ejemplo el `403` de la Clase 7 cuando el usuario es solo `miembro` de ese proyecto, muestra el mensaje que armó nuestro `custom_exception_handler` (Clase 4).

- `...authHeaders()` (los tres puntos) "desparrama" el contenido de un objeto dentro de otro: el resultado es un único objeto con los dos headers.
- Usamos siempre `textContent` (nunca `innerHTML`) para mostrar datos que vienen de la API: así, si una tarea tuviera HTML en su título, el navegador lo muestra como texto y no lo ejecuta. Es la otra mitad de la defensa contra XSS de la Clase 8.

Parte 3 — Servir la página y probar (25 min)

3.1 Por qué no abrirla con doble clic

Concepto: sitios estáticos. Esta página es un **sitio estático**: archivos HTML, CSS y JavaScript que el servidor entrega tal cual, sin procesarlos. Por eso se puede servir con algo tan simple como `http.server`, y publicar gratis en plataformas como Vercel, Netlify o Cloudflare Pages, que los distribuyen desde una **CDN** (*Content Delivery Network*): una red de servidores repartidos por el mundo que entregan cada archivo desde el lugar más cercano al usuario. Toda la lógica y los datos siguen en la API.

Práctica.

Abrir `index.html` con doble clic usa el esquema `file://`. En ese caso el navegador manda el origen especial `null`, que no está en nuestra lista de CORS, y además algunos navegadores bloquean `fetch` desde archivos locales. Mejor: sírvanla con un servidor HTTP simple.

Linux/Mac/Windows (con Python, que ya tienen instalado desde la Clase 1): parados en `taskflow-frontend/`, en una terminal aparte (no hace falta el entorno virtual: es un módulo de la librería estándar de Python):

```
python3 -m http.server 5500 --bind 127.0.0.1
```

(En Windows, `python` en vez de `python3` si así lo tienen configurado.) Abran `http://127.0.0.1:5500/` en el navegador.

Alternativa en VS Code: instalen la extensión gratuita **"Live Server"** y usen "Open with Live Server" con el botón derecho sobre `index.html`. Live Server usa el puerto 5500 por defecto, pero puede abrir con `localhost` en vez de `127.0.0.1`: si es así, cambien la dirección en la barra del navegador o agreguen `http://localhost:5500` a `CORS_ALLOWED_ORIGINS`.

3.2 Probar el flujo completo

- Ingresen con un usuario ya registrado (Clase 6).
- Verifiquen que aparezca el listado de tareas de los proyectos donde ese usuario es miembro, y el desplegable con esos proyectos (Clase 7).
- Elijan un proyecto donde el usuario sea **admin** y agreguen una tarea: aparece arriba de la lista (el repositorio de la Clase 5 ordena de la más nueva a la más vieja).
- Elijan un proyecto donde sea solo **miembro** e intenten agregar una: aparece en rojo el mensaje del `403`.
- Si algo no funciona, abran las herramientas de desarrollador del navegador (`F12`) → pestaña **Console** o **Network**: ahí se ve cada petición con su código de estado. Un error que menciona `CORS policy` significa que el origen no está en `CORS_ALLOWED_ORIGINS` (Parte 1.1).

Parte 4 — Guardar en Git y GitHub (10 min)

Este es un proyecto **aparte** de `taskflow/` (otra carpeta), así que necesita su propio repositorio. Parados en `taskflow-frontend/` (ya están ahí desde la Parte 2.1):

```
git init
git add .
git commit -m "Frontend minimo de demostracion para TaskFlow"
```

Creen un repositorio nuevo en GitHub (`taskflow-frontend`) y súbanlo con el mismo flujo de la Clase 1, Parte 4.5. En el repositorio del backend, no olviden commitear el cambio de `CORS_ALLOWED_ORIGINS`.

Parte 5 — Tarea opcional

- Guarden el token en `sessionStorage` para que no se pierda al recargar la página, y agreguen un botón "Salir" que lo borre.
- Agreguen botones "Anterior" / "Siguiente" usando los campos `next` y `previous` de la respuesta paginada.
- Conecten la página al WebSocket de la Clase 10 (`new WebSocket("ws://127.0.0.1:8000/ws/projects/" + id + "/?token=" + accessToken)`) y recarguen la lista cada vez que llegue un evento `task_created`: van a ver aparecer las tareas que crea otro usuario, sin refrescar.

Preguntas de repaso

1. ¿Qué ventajas tiene separar el frontend del backend? ¿Quién más, además de esta página, consume la API de TaskFlow?
2. Comparen el renderizado en el servidor con una SPA.
3. ¿Por qué `fetch` es asincrónico? ¿Qué relación tiene con el `async`/`await` de Python de la Clase 9?
4. ¿Por qué hay que revisar `response.ok` después de un `fetch`?
5. ¿Qué es el DOM? ¿Por qué usamos `textContent` y no `innerHTML` para mostrar datos de la API?
6. ¿Por qué la página no funciona bien abierta con doble clic?
7. ¿Qué es un sitio estático y qué es una CDN?

Para profundizar

- MDN, *Usando Fetch*: `developer.mozilla.org/es/docs/Web/API/Fetch_API/Using_Fetch`
- MDN, *async y await*: `developer.mozilla.org/es/docs/Learn/JavaScript/Asynchronous/Promises`
- MDN, *Introducción al DOM*: `developer.mozilla.org/es/docs/Web/API/Document_Object_Model/Introduction`

Diccionario de términos de esta clase

- **CDN (Content Delivery Network):** red de servidores distribuidos que entregan archivos desde el punto más cercano al usuario.
- **DOM:** representación del HTML de una página como objetos que JavaScript puede leer y modificar.
- **`fetch`:** función nativa de JavaScript para hacer peticiones HTTP desde el navegador.
- **Frontend:** la parte de la aplicación que corre en el navegador del usuario (repasar Clase 1, Parte 1.4).
- **`http.server`:** módulo de la librería estándar de Python que sirve una carpeta como sitio web simple, sin instalar nada.
- **Live Server:** extensión gratuita de VS Code que sirve archivos HTML locales por HTTP, con recarga automática.
- **Promesa (Promise):** objeto de JavaScript que representa un resultado que todavía no está disponible.
- **Sitio estático:** conjunto de archivos que el servidor entrega sin procesar.
- **SPA (Single Page Application):** aplicación web que carga una sola página y actualiza la pantalla con JavaScript, pidiendo datos a una API.
- **`textContent`:** propiedad de los elementos HTML que asigna texto plano, sin interpretar etiquetas HTML.